

WAOW Admin — Backend Architecture Guide

For presentation / oral defense

Repository: <https://github.com/souksanDev1/waow-admin-task>

English + structured explanation of folders and code flow

1. What the backend is

The backend is a **REST Admin API** built with:

- **Node.js + Express** — HTTP server
- **PostgreSQL + Sequelize** — database and ORM
- **JWT** — admin authentication
- **RBAC** — role-based permissions (dynamic JSON permissions)
- **pnpm** — package manager

API base path: `/api/v1`

Default port: `3002`

Shared database with User API: Postgres on `localhost:5434`, database `waow_backend`

Main features

1. Admin login (username + password → JWT)
 2. Role management (dynamic permissions)
 3. Admin CRUD with **soft delete**
 4. Reset password (SUPER_ADMIN only)
 5. List users from the shared `users` table (read-only)
-

2. Request flow (most important to remember)

Every protected API request follows this path:

```
Client (Frontend / Postman)
  → routes          (URL + HTTP method)
  → middlewares     (validate, JWT, RBAC)
  → controllers     (thin HTTP layer)
  → services        (business logic)
  → models          (Sequelize ↔ PostgreSQL)
  → response JSON   { error, code, message, data }
```

If something fails, `errorHandler` converts it into the standard error envelope.

Interview tip: Say that controllers stay thin; **services** own the business rules and transactions.

3. Top-level `backend/` folder

Path	Purpose
<code>package.json</code>	Scripts (<code>dev</code> , <code>db:migrate</code> , <code>db:seed</code> , <code>lint</code>) and dependencies
<code>.env</code> / <code>.env.example</code>	Port, JWT secret, DB connection (not committed for <code>.env</code>)
<code>.sequelizerc</code>	Tells sequelize-cli where config / models / migrations / seeders live
<code>eslint.config.js</code>	Lint rules
<code>src/</code>	All application source code

4. Entry points

`src/server.js`

- Loads Express app
- Connects to the database with `sequelize.authenticate()`
- Starts listening on `PORT` (3002)

This is what `pnpm dev` / `nodemon` runs.

`src/app.js`

- Creates the Express application
- Enables `CORS`, JSON body parsing
- Mounts all routes under `/api/v1`
- Handles unknown routes (404)
- Attaches the global `errorHandler`

5. `src/config/` – configuration

File	Role
<code>env.js</code>	Reads environment variables (port, JWT, DB) in one place
<code>database.js</code>	Sequelize / sequelize-cli connection settings for development / test / production

Keeps secrets and environment values out of business code.

6. `src/routes/` — URL map

Routes answer: **Which URL goes where, and which middlewares apply?**

File	Responsibility
<code>index.js</code>	Combines all routers + GET <code>/health</code>
<code>auth.routes.js</code>	POST <code>/auth/login</code> (public)
<code>role.routes.js</code>	<code>/roles</code> — requires JWT + SUPER_ADMIN
<code>admin.routes.js</code>	<code>/admins</code> — requires JWT + <code>admin_module</code> permissions
<code>user.routes.js</code>	GET <code>/users</code> — requires JWT + <code>user_module.can_view</code>

Important endpoints

Method	Path	Access
POST	<code>/api/v1/auth/login</code>	Public
CRUD	<code>/api/v1/roles</code>	SUPER_ADMIN
CRUD	<code>/api/v1/admins</code>	Permission-based
POST	<code>/api/v1/admins/:id/reset-password</code>	SUPER_ADMIN
GET	<code>/api/v1/users</code>	<code>user_module.can_view</code>
GET	<code>/api/v1/health</code>	Public

7. `src/middlewares/` — security and validation gates

File	What it does
<code>validate.js</code>	Validates request body with Joi; rejects unknown fields (<code>VALIDATION_ERR_UNKNOWN_FIELD</code>)
<code>authenticate.js</code>	Reads <code>Authorization: Bearer <token></code> , verifies JWT, loads admin + role into <code>req.auth</code>
<code>authorize.js</code>	<code>requireSuperAdmin</code> and <code>requirePermission(module, action)</code> for RBAC
<code>errorHandler.js</code>	Maps errors to <code>{ error: true, code, message, data }</code>

Interview tip: Authentication = “who are you?” · Authorization = “what are you allowed to do?”

8. `src/validations/` — input schemas

`schemas.js`

Joi schemas for:

- Login (`username` , `password`)
- Create / update role (`name` , `permissions` JSON shape)
- Create / update admin
- Reset password

All schemas use `.unknown(false)` so extra fields are rejected (per assignment requirements).

9. `src/controllers/` — HTTP adapters

Controllers:

1. Read `req.body` / `req.params` / `req.auth`
2. Call the matching **service**
3. Return `success(res, data, message)`

They do **not** contain heavy business logic.

File	Handles
<code>auth.controller.js</code>	Login
<code>role.controller.js</code>	Role list / create / update / delete
<code>admin.controller.js</code>	Admin list / create / update / soft delete / reset password
<code>user.controller.js</code>	User list

10. `src/services/` — business logic (core)

This is the most important folder for explaining the product.

File	Responsibility
<code>auth.service.js</code>	Find non-deleted admin, compare password hash, sign JWT, return profile + permissions
<code>role.service.js</code>	CRUD roles; protect system roles <code>SUPER_ADMIN</code> and <code>NORMAL</code> ; block delete if role still assigned

admin.service.js	CRUD admins (transactions when needed); soft delete; reset password
user.service.js	Read-only list from shared users table

Soft delete (admins)

When deleting an admin:

- Set `is_deleted = true`
- Set `deleted_at = now`
- Do **not** remove the row from the database
- Soft-deleted admins cannot log in and are excluded from normal lists

Transactions

Admin create/update uses Sequelize transactions so partial writes do not leave inconsistent data.

11. `src/models/` — database mapping

File	Table / meaning
index.js	Loads models, runs associations, exports sequelize
user.js	users (shared with User API; read-only here)
role.js	roles with JSONB permissions
admin.js	admins with soft-delete fields + <code>toSafeJSON()</code> (never returns password hash)

Associations

- `Admin.belongsTo(Role)`
- `Role.hasMany(Admin)`

Permissions JSON example

```
{
  "admin_module": {
    "can_create": false,
    "can_delete": true,
    "can_update": true,
    "can_view": true
  },
  "user_module": {
    "can_create": true,
    "can_delete": false,
```

```
"can_update": true,  
"can_view": true  
}  
}
```

12. `src/migrations/` — schema evolution

Run with: `pnpm db:migrate`

Migration	Effect
<code>...create-users.js</code>	Creates users only if missing (skips when User API already created it)
<code>...create-roles.js</code>	Creates roles
<code>...create-admins.js</code>	Creates admins with foreign key to roles

This supports the requirement: **same database as the User API project**.

13. `src/seeder/` — initial data

`default-roles-and-superadmin.js`

Run with: `pnpm db:seed`

Creates (if missing):

1. Role `SUPER_ADMIN` — full permissions
2. Role `NORMAL` — view only (no create/update/delete)
3. Admin `superadmin / 123456` (bcrypt hashed) assigned to `SUPER_ADMIN`

14. `src/utls/` — shared helpers

File	Role
<code>response.js</code>	Standard success / fail JSON envelope
<code>errors.js</code>	<code>AppError</code> with HTTP status + error code
<code>jwt.js</code>	Sign and verify JWT
<code>password.js</code>	bcrypt hash and compare

Response format (assignment requirement)

Success:

```
{
  "error": false,
  "code": 0,
  "message": "Success",
  "data": {}
}
```

Failure:

```
{
  "error": true,
  "code": "ADMIN_ERR_001",
  "message": "Error message here",
  "data": {}
}
```

15. How to explain in 60 seconds (script)

"Backend is Express with clean architecture.

Routes map URLs. Middlewares handle validation, JWT authentication, and RBAC authorization.

Controllers are thin. Services contain business logic such as soft delete and transactions.

Models map to PostgreSQL tables `users`, `roles`, and `admins`.

We share the same database as the User API: we reuse `users` and add admin/role tables.

Default superadmin is seeded for login, and every response uses a standard JSON envelope."

16. Likely defense questions & short answers

Q: Why separate controllers and services?

A: Controllers handle HTTP only; services keep business rules reusable and testable.

Q: How does RBAC work?

A: Each role stores permissions JSON. Middleware checks `admin_module` / `user_module` flags. SUPER_ADMIN bypasses checks.

Q: What is soft delete?

A: Admin rows are marked deleted with `is_deleted` and `deleted_at`, not physically removed.

Q: Why share the User API database?

A: Assignment requires the same database so admin can list real users from the User API `users` table.

Q: How is password stored?

A: bcrypt hash only; never returned in API responses.

Q: What happens with unknown request fields?

A: Joi validation rejects them with `VALIDATION_ERR_UNKNOWN_FIELD` .

17. Project map (backend only)

```
backend/
|-- package.json
|-- .env.example
|-- .sequelizerc
`-- src/
    |-- server.js           Start process
    |-- app.js             Express app wiring
    |-- config/            Env + DB config
    |-- routes/            URL mapping
    |-- middlewares/       Auth, RBAC, validate, errors
    |-- validations/       Joi schemas
    |-- controllers/       HTTP layer
    |-- services/          Business logic
    |-- models/            Sequelize models
    |-- migrations/        SQL schema changes
    |-- seeders/           Default roles + superadmin
    `-- utils/             Response, JWT, password, AppError
```

Repository: <https://github.com/souksanDev1/waow-admin-task>

END OF BACKEND ARCHITECTURE GUIDE